STUDY PROJECT — PHASE 3

Operational Complexity Meter

Quantifying Operational Complexity in Distributed Systems

Aabid Ali Sofi | 2023EBCS041
Advisor: Preethy P Johny | Feb 2026

Agenda

1. Problem Statement & OCM Overview
2. The 6 Metrics & Composite Score
3. System Architecture & Pipeline
4. Parser, Graph Algorithms
1. Storage & REST API
2. Dashboard & Visualization
3. Testing & Validation
4. Limitations, Future Work & Summary

Problem Statement

Symptoms

- Hundreds of config parameters
- Deep dependency chains
- Frequent config changes
- Environment drift (dev vs prod)
- Exposed failure surfaces

Impact

- Outages from config errors
- Slow incident response
- Hidden coupling between services
- Unmeasured toil & risk

OCM — The Solution

A **single Go binary** that:

1. **Scans** a Kubernetes YAML repository
2. **Computes** 6 complexity metrics per service
3. **Produces** a composite OCM score (0–1)
4. **Stores** results in SQLite
5. **Serves** an interactive web dashboard

```
go run ./cmd/ocm --repo ./k8s-repo --db ocm.sqlite --port 8080
```

Can we quantify operational complexity as a single, measurable score?

The 6 Metrics

Metric	What It Measures	How
CSA	Configuration Surface Area	Count of env vars, ports, resources, replicas, spec keys
DD	Dependency Depth	Longest path in SCC-condensed dependency DAG
DB	Dependency Breadth	Direct upstream + downstream dependency count
CV	Change Volatility	Git commits affecting config in last 30 days
FE	Failure Exposure	Exposed endpoints (LB, NodePort, Ingress, hostPort, ext URLs)
CDR	Config Drift Risk	Env-specific overrides (dev/staging/prod directory diffs)

Normalization (Min-Max)

$$\mathrm{Norm}(M) = \frac{M - M_{\min}}{M_{\max} - M_{\min}}$$

Composite OCM Score

$$\mathrm{OCM} = \frac{1}{6} \sum_{m \in M} \mathrm{Norm}(m)$$

Metric Formulas

CSA

$$\mathrm{CSA}(s) = |\mathrm{env}| + |\mathrm{ports}| + |\mathrm{resources}| + |\mathrm{replicas}| + |\mathrm{spec\ keys}|$$

DD

$$\mathrm{DD}(s) = \max_{p \in \text{ paths}} |p| \quad (\text{SCC DAG})$$

DB

$$\mathrm{DB}(s) = \deg^{+}(s) + \deg^{-}(s)$$

CV

$$\mathrm{CV}(s) = |\{c : \mathrm{touches}(c, s) \wedge c \in [t - \Delta, t]\}|$$

FE

$$\mathrm{FE}(s) = |\mathrm{LB}| + |\mathrm{NP}| + |\mathrm{Ingress}| + |\mathrm{hostPorts}| + |\mathrm{ext\ URLs}|$$

CDR

$$\mathrm{CDR}(s) = \sum_{f} (|\mathrm{envs}(f)| - 1)^{+}$$

0.0 Minimal complexity

Weights **configurable**
(default: equal 1/6)

1.0 Maximum in cohort

System Architecture

```
┌──────────────────────────────────┐
│       ocm (single binary)        │
│                                  │
│  ┌─────┐   ┌────────┐   ┌──────┐ │
│  │ CLI │──▶│Pipeline│──▶│SQLite│ │
│  └─────┘   │        │   └──────┘ │
│            │ Parser │      │     │
│            │ GitLog │      ▼     │
│            │ Metrics│  ┌──────┐  │
│            │ Score  │  │HTTP API│ │
│            └────────┘  │+Dashbd │ │
│                        └──────┘  │
└──────────────────────────────────┘
```

2 Dependencies Only

`yaml.v3` + `modernc.org/sqlite`

No CGO

Pure Go SQLite, cross-compilable

Embedded Assets

`go:embed` HTML/CSS/JS — zero external files

7 Packages

model, parser, pipeline, gitlog, storage, api, dashboard

Data Flow Pipeline

1. **Discovery** — Walk filesystem, collect `.yaml`/`.yml`
2. **Parsing** — Multi-doc YAML decode, extract CSA + deps + FE
3. **Service ID** — Map files to services via `dir` or `manifest`
4. **Metric Engine** — Compute DD, DB, CV, FE, CDR
5. **Normalization** — Min-max across cohort
6. **Composite** — Weighted sum to single OCM score
7. **Persist** — Atomic transaction to SQLite
8. **Serve** — HTTP API + embedded dashboard

Parser & Extraction

CSA — Config Surface Contributors

Component	Source
`env`	Container environment variables
`port`	Container port entries
`resource`	Resource limits/requests keys
`replica`	`spec.replicas` field
`spec_key`	Top-level spec keys

FE — 5 Detection Rules

- LoadBalancer / NodePort ports
- Ingress `paths[]` entries
- ExternalName services
- Container `hostPort` bindings
- External URLs in env vars

Dependency Detection — 3 Heuristics

Rule 1 — Env var suffix matching

`_SERVICE`, `_HOST`, `_ADDR`, `_URL`, `_ENDPOINT`, `_URI`

Rule 2 — Cluster FQDN

Values containing `.svc.cluster.local`

Rule 3 — Hostname:port regex

`^[a-z][a-z0-9-]{0,62}:\d{2,5}$`

CDR — Config Drift Risk

Counts env-specific overrides across: dev, staging, prod, qa, test, uat, preview, canary, local

Graph Algorithms — Dependency Depth

Problem

Dependency graphs may have cycles (A → B → A).

Solution — SCC Condensation

1. Tarjan's algorithm finds SCCs
2. Condense each SCC to a single node
3. Result is a DAG (no cycles)
4. DFS with memoization → longest path

Example

```
A → B → C     (B ↔ A = cycle)
    ↑ ↙
    A

SCC: {A, B} → node S₁
DAG: S₁ → C
DD[A] = DD[B] = 1,  DD[C] = 0
```

Deterministic — same input always produces same DD values.

Storage & REST API

SQLite Schema — 4 Tables

```
services(id, name, repository)
metrics(id, service_id, metric_type,
        metric_value, timestamp)
composite_scores(id, service_id,
        ocm_score, timestamp)
metric_evidence(id, service_id,
        metric_type, component,
        evidence_key, evidence_value,
        source_path, manifest_kind,
        manifest_name, timestamp)
```

FK constraints + indexes
Atomic transactions

JSON responses
CORS middleware

REST API — 6 Endpoints

Endpoint	Returns
`/api/healthz`	Health check
`/api/services`	All services
`/api/overview`	Repo aggregates
`/services/{id}/scores`	Score time series
`/services/{id}/metrics/{t}`	Metric series
`/services/{id}/.../evidence`	Evidence items

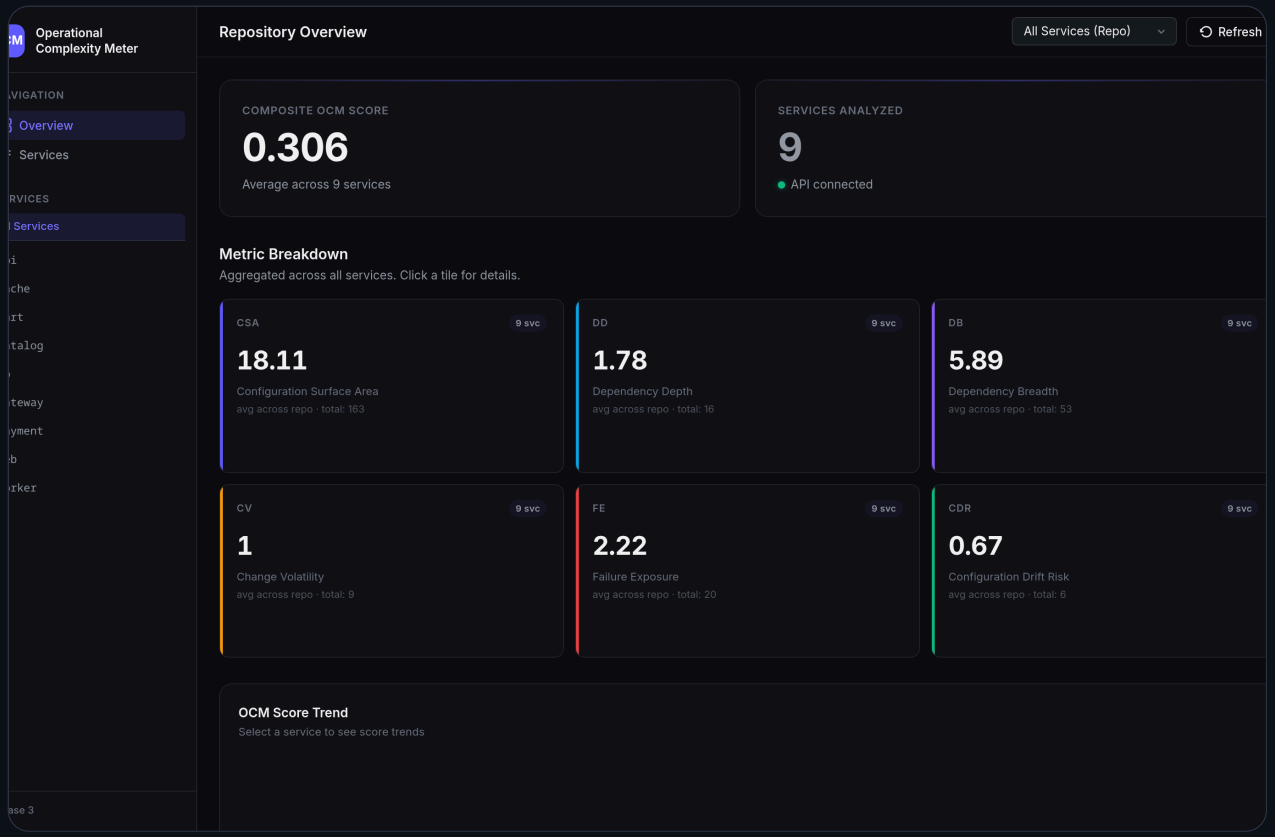
Dashboard — Design & Overview

Views & Charts

- Composite OCM score hero card
- 6 metric tiles with color coding
- OCM trend chart (Canvas)
- Radar chart + Bar chart
- Service comparison table
- Evidence drill-down modal

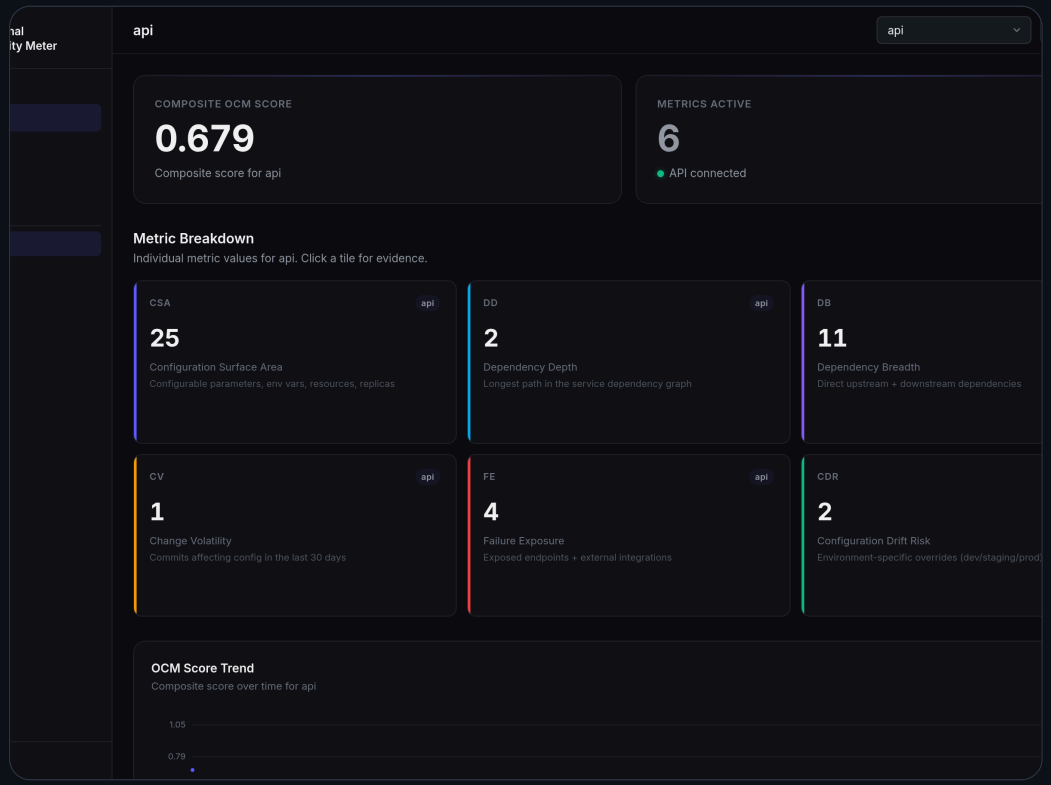
Tech Stack

- Stripe-inspired dark theme
- Pure HTML/CSS/JS (no frameworks)
- Canvas-based chart rendering
- Responsive + embedded via `go:embed`

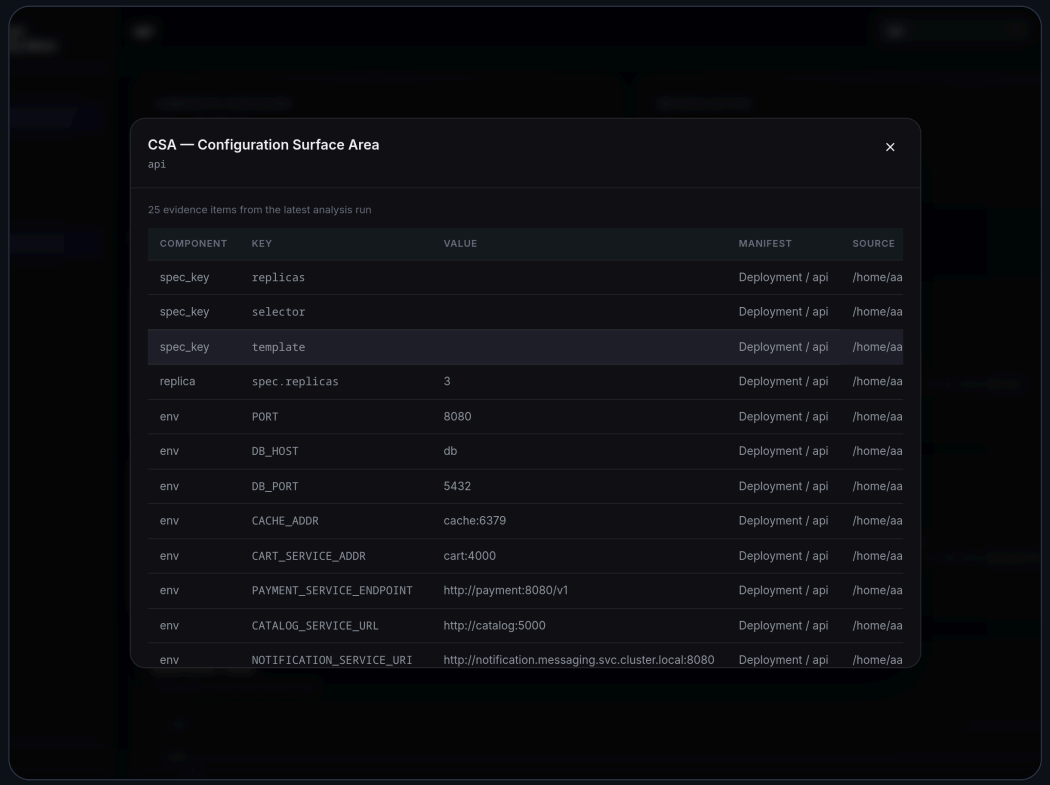


Repository overview — composite score, 9 services, 6 metric tiles

Dashboard — Service Detail & Evidence



Service detail view



COMPONENT	KEY	VALUE	MANIFEST	SOURCE
spec_key	replicas		Deployment / api	/home/aa
spec_key	selector		Deployment / api	/home/aa
spec_key	template		Deployment / api	/home/aa
replica	spec.replicas	3	Deployment / api	/home/aa
env	PORT	8080	Deployment / api	/home/aa
env	DB_HOST	db	Deployment / api	/home/aa
env	DB_PORT	5432	Deployment / api	/home/aa
env	CACHE_ADDR	cache:6379	Deployment / api	/home/aa
env	CART_SERVICE_ADDR	cart:4000	Deployment / api	/home/aa
env	PAYMENT_SERVICE_ENDPOINT	http://payment:8080/v1	Deployment / api	/home/aa
env	CATALOG_SERVICE_URL	http://catalog:5000	Deployment / api	/home/aa
env	NOTIFICATION_SERVICE_URI	http://notification.messaging.svc.cluster.local:8080	Deployment / api	/home/aa

Evidence drill-down

Evidence-Based Traceability

Metric	Evidence Type	Example
CSA	`env`, `port`, `resource`, `replica`	`PORT=8080` from `api/deploy.yaml`
DD	`dep_chain`, `direct_dep`	`api -> cache -> db` (depth=2)
DB	`upstream_dep`, `downstream_dep`	`orders -> payment` edge
CV	`commit`	`commit abc12345`
FE	`loadbalancer`, `ingress`, `external_url`	LoadBalancer port 80
CDR	`env_override`	`config.yaml` in dev + prod

Every metric value links to specific YAML files, manifest keys, and configuration parameters.

Testing & Validation

Parser Tests (16)

- 6 FE detection tests
- 8 dependency extraction tests
- CronJob, FQDN, regex patterns
- Numeric value rejection

Pipeline Tests (8)

- DD cycle handling (Tarjan's)
- DB in/out degree counting
- Self-dependency ignored
- Normalization edge cases

Storage & Git (8)

- Round-trip all 6 metric types
- 6 git log parsing tests
- Empty output handling
- Graceful degradation

All 32 tests pass | Build clean | Vet clean

Unit: Isolated function tests (normalization, extraction)

Integration: Full pipeline with temp K8s repos

Persistence: Round-trip in-memory SQLite

Phase Evolution & Limitations

Phase 2 — MVP

- 2 metrics: CSA + DD
- Basic dependency extraction
- Simple dashboard
- CSA evidence only
- 50/50 composite weights

Phase 3 — Complete

- All 6 metrics implemented
- 9 dependency patterns
- Git integration for CV
- Evidence for all metrics
- Redesigned dashboard
- 32 automated tests

Known Limitations

- No Helm template rendering
- Heuristic-based extraction
- Single-run model
- `git` binary needed for CV
- Localhost only (no auth)

Mitigations Evidence trail for verification | Multiple service ID strategies | Best-effort approach | Modular architecture for extension

Future Work & Summary

Near-Term Extensions

- Helm v3 template rendering
- Watch mode / scheduled re-analysis
- Configurable weights via UI
- Export reports (PDF/CSV)

Long-Term Vision

- CI/CD integration (PR complexity diffs)
- Multi-repo support
- Complexity budget enforcement
- Authentication & multi-user

Key Achievements

+ **6 metrics** quantifying ops complexity
+ **Single binary** — Go + SQLite + dashboard
+ **Evidence-based** — full traceability
+ **Graph algorithms** — Tarjan's SCC
+ **2 dependencies** — yaml.v3 + pure-Go SQLite
+ **Interactive dashboard** — charts & drill-down
+ **32 automated tests** across 4 packages

Thank You

Questions?

Aabid Ali Sofi | 2023EBCS041
Operational Complexity Meter — Phase 3

github.com/aabidsofi19/ocm